

UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

Tecnologías de la Información y la Comunicación



Tema

Actividad de Aprendizaje 3

Integrantes

Andrade Alvarado Eder Jonathan
Cisneros Cárdenas Freddy Gabriel

Asignatura

Arquitectura de Software

NRC

29692

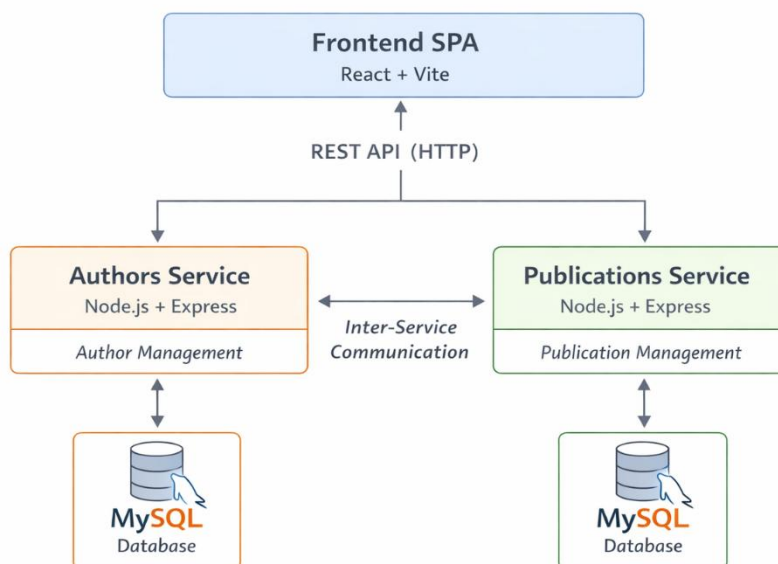
Período

Octubre – Marzo 2025-2026

Repositorio

- <https://github.com/EderJAndrade/29692-actividad3-ederandrade-freddycisneros.git>

Arquitectura del Sistema



El proyecto fue diseñado bajo el estilo Arquitectura de Microservicios, separando responsabilidades de negocio en servicios independientes y desacoplados.

Se compone de:

Frontend (SPA)

- Aplicación web desarrollada con React + Vite
- Consume APIs REST vía HTTP
- No accede directamente a bases de datos

Authors Service

- Gestiona el ciclo de vida de autores
- CRUD de autores
- Base de datos MySQL propia

Publications Service

- Gestiona publicaciones y estados editoriales
- Valida autores consultando Authors Service vía HTTP
- Base de datos MySQL propia

Bases de datos

Cada microservicio tiene su propia base de datos, logrando:

- Aislamiento
- Bajo acoplamiento
- Escalabilidad independiente

Comunicación

- REST HTTP (JSON)
- Sin dependencias circulares
- Publications hacia Authors únicamente

The screenshot shows a web browser window with the address bar displaying `http://localhost:8080`. The page has a blue header bar with the text "Editorial: Autores & Publicaciones" on the left and "AUTORES PUBLICACIONES" on the right. The main content area is divided into two panels. The left panel, titled "Crear autor", contains three input fields: "Nombre", "Email", and "Nombre de pluma (opcional)", followed by a "GUARDAR" button. The right panel, titled "Autores", displays the message "No hay autores aún." and a "RECARGAR" button.

Patrones Aplicados

1. Repository Pattern

El objetivo es separar lógica de acceso a datos de la lógica de negocio.

- authors-service/src/repositories/AuthorsRepository.ts
- publications-service/src/repositories/PublicationsRepository.ts

Utilidad

- Desacoplar ORM del dominio
- Facilitar el testing
- Permite cambiar DB sin afectar servicios

2. Factory Method

Centraliza la creación de tipos de publicación.

- publications-service/src/domain/factory/PublicationFactory.ts

Utilidad

- Evita lógica condicional dispersa
- Creación de Book/Article
- Principio Open/Closed

3. Strategy Pattern

Gestiona reglas de transición de estados editoriales.

- `publications-service/src/domain/status/PublicationStatusMachine.ts`
- `publications-service/src/domain/status/StatusTransitionStrategy.ts`
- `publications-service/src/domain/status/DefaultStatusTransitionStrategy.ts`

Utilidad

- Reglas intercambiables
- Fácil extensión de nuevos estados
- Elimina if/else complejos

Evidencia de SOLID

S: Single Responsibility

- **Controllers:** Manejan las peticiones HTTP y las respuestas del API REST.
- **Services:** Contienen la lógica de negocio y las reglas del dominio.
- **Repositories:** Abstraen el acceso a la base de datos mediante el ORM.
- **Entities:** Modelan las estructuras del dominio y sus relaciones.

Cada clase tiene una única responsabilidad.

O: Open/Closed

- Nuevos tipos de publicación se agregan vía Factory sin modificar código existente
- Nuevas estrategias de estado se agregan implementando Strategy

L: Liskov

- `BookPublication` y `ArticlePublication` heredan de `PublicationBase`.
- Se pueden sustituir sin romper comportamiento

I: Interface Segregation

- DTOs específicos para cada request

- Interfaces pequeñas y enfocadas

D: Dependency Inversion

- Services dependen de repositorios abstractos
- Publications usa AuthorsHttpClient en vez de axios directo

Instrucciones de despliegue y ejecución

Requisitos

- Docker Desktop
- Camunda Modeler

Pasos:

- `git clone https://github.com/EderJAndrade/29692-actividad3-ederandrade-freddycisneros.git`
- `cd 29692-actividad3-ederandrade-freddycisneros`
- `cp .env.example .env`
- `docker compose up --build`

URLs

- **Frontend:** `http://localhost:8080`
- **Authors API:** `http://localhost:3001`
- **Publications API:** `http://localhost:3002`

Health Checks:

- `http://localhost:3001/health`
- `http://localhost:3002/health`

Endpoints Principales

- **Authors Service**

Crear autor

POST http://localhost:3001/authors

Body:

```
{  
  
  "name": "Juan Pérez",  
  
  "email": "juan@email.com",  
  
  "penName": "JP"  
}
```

Listar autores

GET http://localhost:3001/authors

Obtener autor por id

GET http://localhost:3001/authors/1

- **Publications Service**

Crear publicación

POST http://localhost:3002/publications

```
{  
  
  "title": "Mi libro",  
  
  "content": "Contenido...",  
  
  "authorId": 1,  
  
  "type": "BOOK",  
  
  "isbn": "123456",  
  
  "genre": "Tecnología"  
}
```

Listar publicaciones

GET <http://localhost:3002/publications>

Obtener detalle (enriquecido con autor)

GET <http://localhost:3002/publications/1>

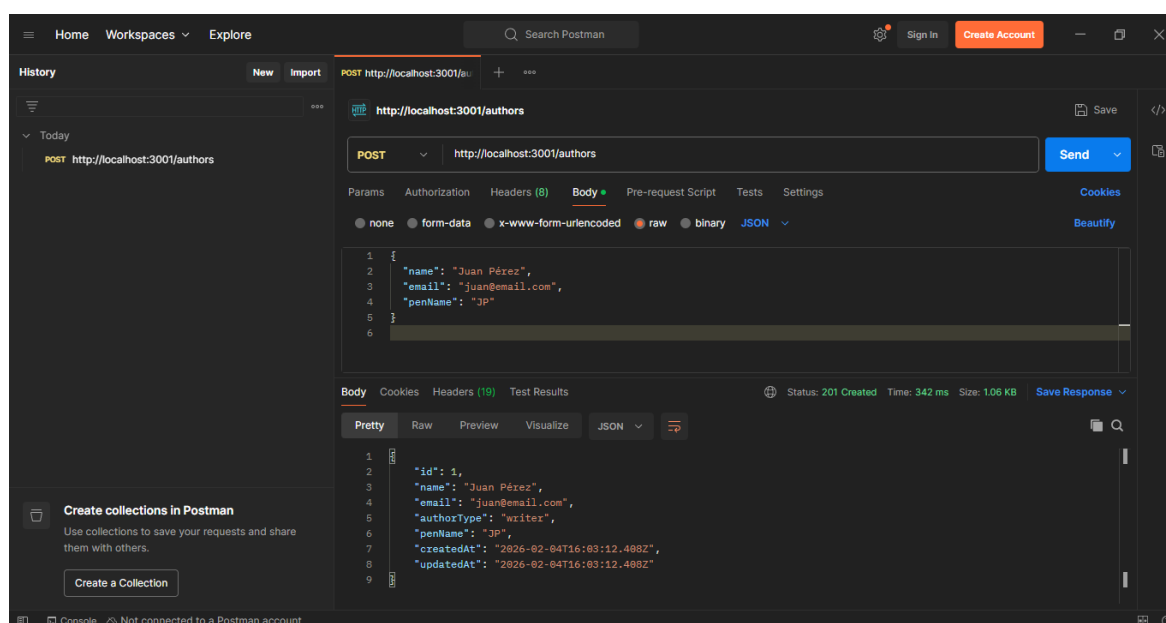
Cambiar estado editorial

PATCH <http://localhost:3002/publications/1/status>

```
{
  "status": "IN_REVIEW"
}
```

Estados permitidos:

- DRAFT
- IN_REVIEW
- APPROVED
- REJECTED
- PUBLISHED



Home Workspaces Explore Search Postman Sign In Create Account

History: Today
 GET http://localhost:3001/authors/1
 GET http://localhost:3001/authors
 POST http://localhost:3001/authors

Create collections in Postman
 Use collections to save your requests and share them with others.
 Create a Collection

POST http://localhost:3001/authors/1 GET http://localhost:3001/authors/1

GET http://localhost:3001/authors/1 Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value

Body Cookies Headers (19) Test Results Status: 200 OK Time: 20 ms Size: 105 KB Save Response

Pretty Raw Preview Visualize JSON

```

1 {
2   "id": 1,
3   "name": "Juan Pérez",
4   "email": "juan@email.com",
5   "authorType": "writer",
6   "penName": "JP",
7   "createdAt": "2026-02-04T16:03:12.408Z",
8   "updatedAt": "2026-02-04T16:03:12.408Z"
9 }

```

Console Not connected to a Postman account

Home Workspaces Explore Search Postman Sign In Create Account

History: Today
 POST http://localhost:3002/publications
 GET http://localhost:3001/authors/1
 GET http://localhost:3001/authors
 POST http://localhost:3001/authors

Create collections in Postman
 Use collections to save your requests and share them with others.
 Create a Collection

POST http://localhost:3002/publications GET http://localhost:3001/authors/1 POST http://localhost:3002/publications

POST http://localhost:3002/publications Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary JSON Beautify

```

1 {
2   "title": "Mi libro",
3   "content": "Contenido...",
4   "authorId": 1,
5   "type": "BOOK",
6   "isbn": "123456",
7   "genre": "Tecnología"
8 }

```

Body Cookies Headers (19) Test Results Status: 201 Created Time: 359 ms Size: 111 KB Save Response

Pretty Raw Preview Visualize JSON

```

1 {
2   "id": 1,
3   "title": "Mi libro",
4   "content": "Contenido...",
5   "status": "DRAFT",
6   "authorId": 1,
7   "type": "BOOK",
8   "extra": {
9     "isbn": "123456",
10    "genre": "Tecnología"
11  },
12   "author": {
13     "id": 1,
14     "name": "Juan Pérez",
15     "email": "juan@email.com",
16     "authorType": "writer",
17     "penName": "JP",
18     "createdAt": "2026-02-04T16:03:12.408Z",
19     "updatedAt": "2026-02-04T16:03:12.408Z"
20   }
21 }

```

Console Not connected to a Postman account

Home Workspaces Explore Search Postman Sign In Create Account

History: Today
 GET http://localhost:3002/publications/1
 POST http://localhost:3002/publications
 GET http://localhost:3001/authors/1
 GET http://localhost:3001/authors
 POST http://localhost:3001/authors

Create collections in Postman
 Use collections to save your requests and share them with others.
 Create a Collection

POST http://localhost:3001/authors/1 GET http://localhost:3001/authors/1 POST http://localhost:3002/publications GET http://localhost:3002/publications/1

GET http://localhost:3002/publications/1 Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value

Body Cookies Headers (19) Test Results Status: 200 OK Time: 43 ms Size: 1.28 KB Save Response

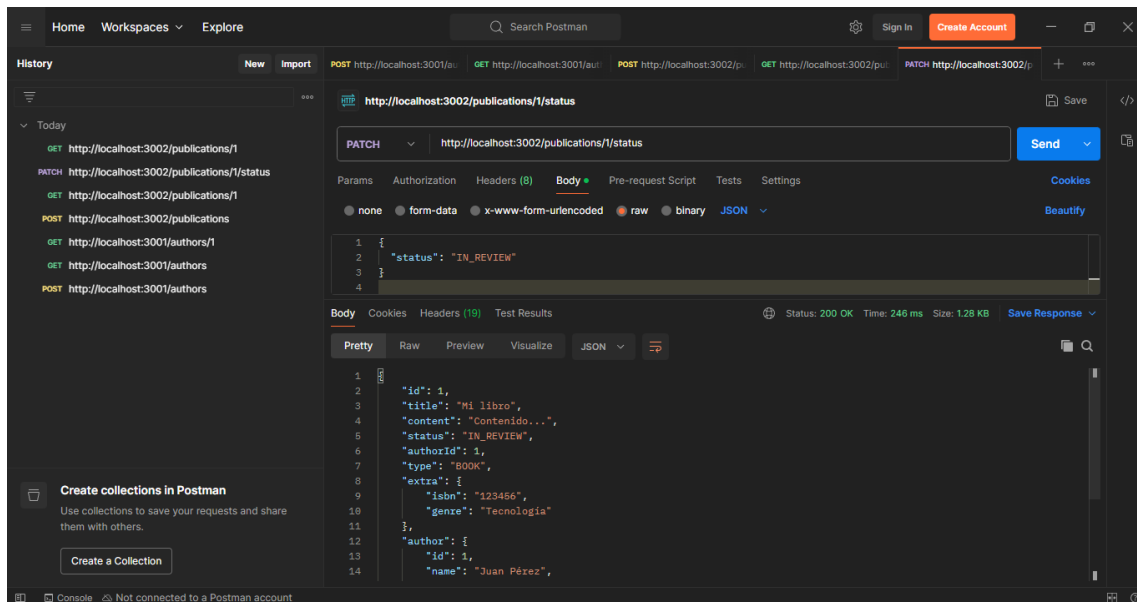
Pretty Raw Preview Visualize JSON

```

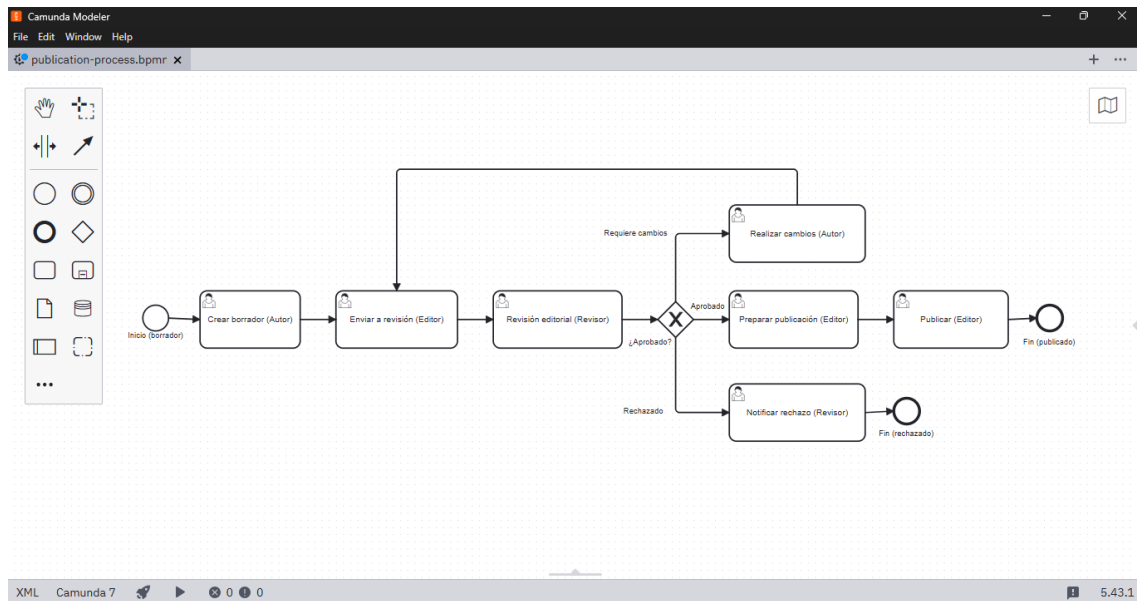
1 {
2   "id": 1,
3   "title": "Mi libro",
4   "content": "Contenido...",
5   "status": "DRAFT",
6   "authorId": 1,
7   "type": "BOOK",
8   "extra": {
9     "isbn": "123456",
10    "genre": "Tecnología"
11  },
12   "author": {
13     "id": 1,
14     "name": "Juan Pérez",
15     "email": "juan@email.com",
16     "authorType": "writer",
17     "penName": "JP",
18     "createdAt": "2026-02-04T16:03:12.408Z",
19     "updatedAt": "2026-02-04T16:03:12.408Z"
20   }
21 }

```

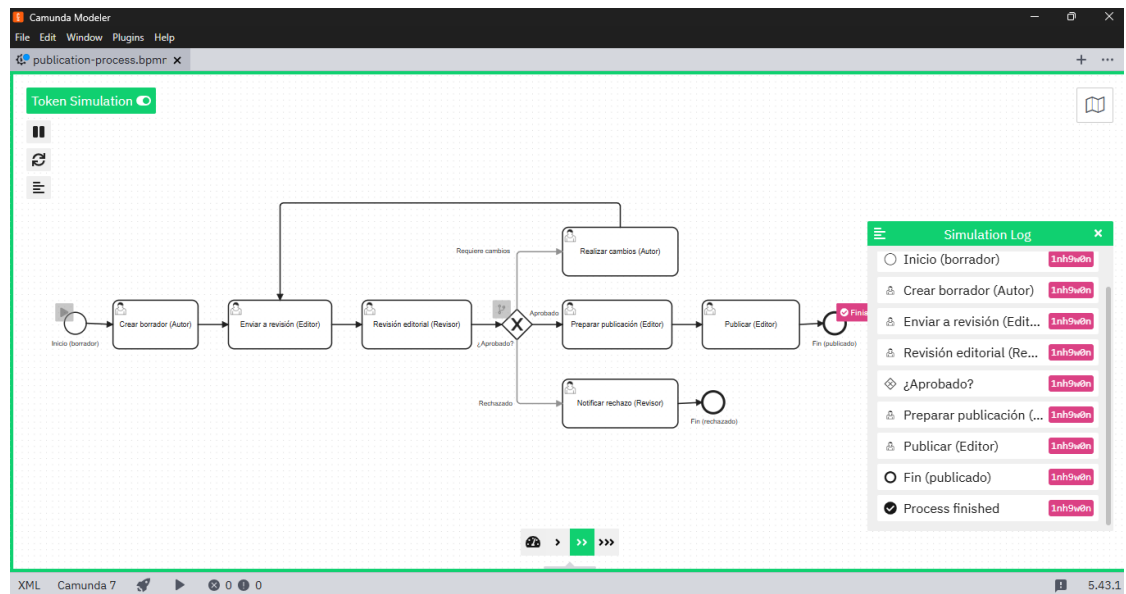
Console Not connected to a Postman account



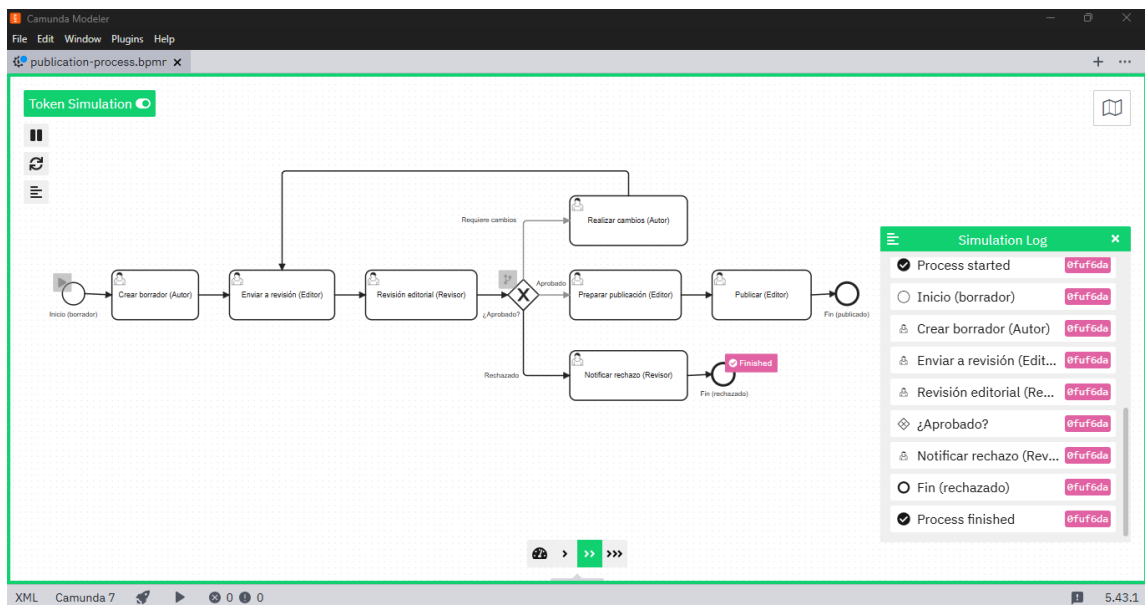
Simulación BPMN



Escenario 1: Publicado



Escenario 2: Rechazado



Escenario 3: Requiere cambios y aceptado

